window is where the codes are written and run en masse, however this part is not important to our tutorial. The top right window maintains a history of all the commands written. The bottom left window is the 'console' where you enter commands and see the results (if any) in the bottom right window.

Select and read data

Let's now create a visualisation. Here's some data from Sachin Tendulkar's batting record each year (Source: stats.espncricinfo.com) pasted in an Excel sheet as shown here:

```
Click on File > Save as > Save it in the CSV (Comma Separated values) format in My Documents (the default location from which RStudio inputs files)
```

In the RStudio console (bottom left), type the following:

```
sachin<-read.csv('sachin.csv',header = TRUE);
```

This will ensure that sachin is now a data set with rows and columns. Type 'sachin' and press [Enter] to view the table with its headers.

The default packages allow some plotting in R. For example, type the following:

```
qplot(sachin$Grouping,sachin$Average);
```

You'll find a plot of the average scores by Sachin in each year, with the 'Grouping' column showing along X-axis and 'Average' column showing along the Y-axis. It might seem a little haphazard so change the scales until you find a graph that works for you. For example:

```
qplot(sachin$Grouping, sachin$Average, ylim = c(0,200));
qplot(sachin$Grouping, sachin$Average, ylim = c(0,2000));
```

These produce radically different visualisations compared to the first data set!

To unleash the full potential of R's graphing capabilities, employ the ggplot2 package. Ensure you have a working internet connection and type:

```
install.packages("ggplot2");
library(ggplot2);
```

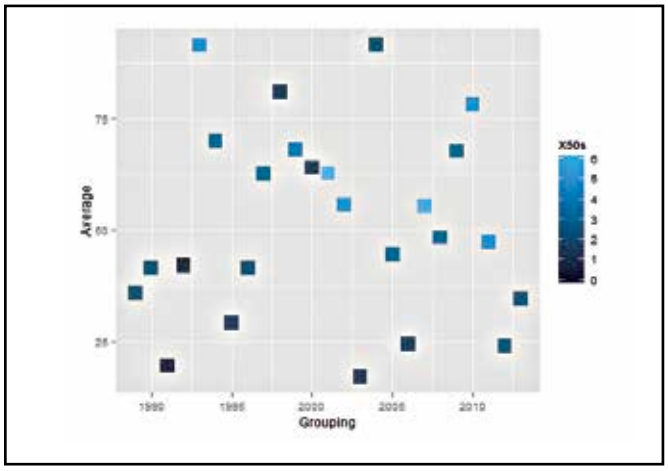
You can now use all the functionalities of ggplot2.

ggplot2 has two basic components: aesthetics, which is all the data and elements, which is the kind of visualisations you want – viz bars, lines, heat maps, etc.

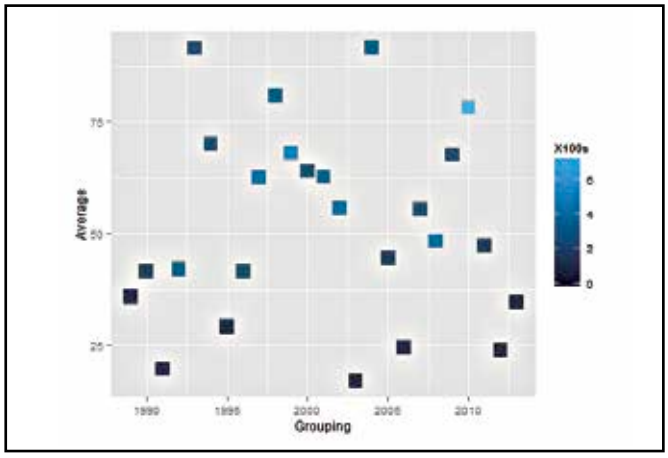
In the console, let's try the following:

```
ggplot(sachin, aes(Grouping, Average)) + geom_point();
```

This yields the same graph as the one shown above with the difference being the ability to customise. Type the following:



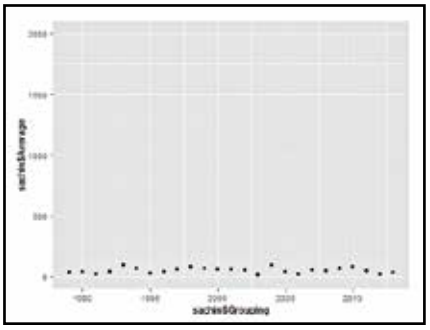
Tendulkar's average annual runs tally shaded according to number of 50s scored



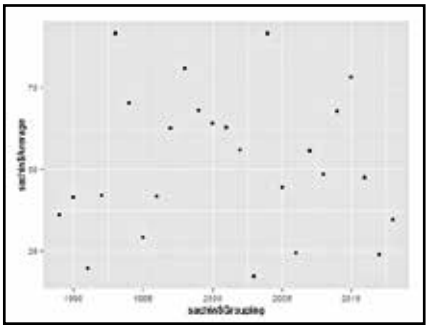
Tendulkar's average annual runs tally shaded according to number of 100s scored

```
ggplot(sachin, aes(x=Grouping,y=Average,colour = X100s))+geom_point(shape=15, size = 6, alpha = 0.8)
```

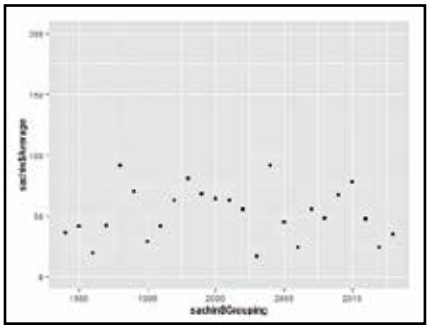
This gives a plot of averages versus grouping, except each point's shade is determined by the hundreds – lighter the shade, more the number of centuries that year. The shape and size can be changed accordingly, and the parameter 'alpha' determines the intensity of colour. Replace 'X100s' with 'X50s' and you'll get the point shaded according to the 50s scored.



Average annual score plotted versus every year - heavily scaled up graph. While there seems to be remarkable consistency, it does appear to push the average score closer to zero



Average annual score plotted versus every year - the default setting



Average annual score plotted versus every year - slightly scaled up graph. Notice how everything seems to appear a lot more 'averaged' and 'flattish'